

▼ Project 5: Implementing an Item-Based Recommender System Using Apache Spark

Introduction

In my previous projects, I built recommender systems using traditional Python libraries to explore how collaborative filtering can generate personalized movie recommendations. While those approaches worked well for a relatively small synthetic dataset, they relied on processing all of the data on a single computer.

The goal of this project is to adapt my Item-Based Collaborative Filtering recommender to Apache Spark. Rather than changing the recommendation strategy, I am changing the platform used to process the data. Spark was designed to distribute large workloads across multiple computers, making it a valuable tool for applications that must process much larger datasets than those used in this course.

Throughout this project, I will compare the Spark implementation with my previous implementation while discussing the advantages, added complexity, and situations where a distributed computing platform becomes beneficial.

Project Design Choice

For this project, I chose to adapt the Item-Based Collaborative Filtering recommender that I developed in Project 4. I selected this approach because I was already familiar with how the recommendation process worked, allowing me to focus on learning how Apache Spark processes data rather than learning a new recommendation algorithm.

My primary objective is not to improve the recommendation algorithm itself, but to evaluate how the same recommender can be implemented using a distributed computing framework. This approach allows me to compare the traditional Python implementation with the Spark implementation while discussing the advantages, added complexity, and situations where using Spark would become beneficial.

▼ Create the Spark Session

Before building the recommender system, I first need to create a Spark session. The Spark session serves as the entry point for working with Apache Spark and allows me to load data, perform transformations, and execute computations using Spark's distributed processing framework.

Although this project uses a relatively small synthetic dataset, creating a Spark session follows the same process that would be used when working with much larger datasets in a distributed environment.

```
# Install PySpark
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.3)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0
```

```
from pyspark.sql import SparkSession

# Create Spark Session
spark = SparkSession.builder \
    .appName("Project5_RecommenderSystem") \
    .getOrCreate()
```

```
# Verify Spark is running
spark
```

SparkSession - in-memory

SparkContext

[Spark UI](#)

Version

v4.0.3

Master

local[*]

AppName

Project5_RecommenderSystem

```
# Check Spark version
print("Spark version:", spark.version)
```

```
Spark version: 4.0.3
```

Interpretation

The Spark session was created successfully, and the version check shows that I am using Spark 4.0.3. This confirms that the Colab environment is ready for the Spark-based recommender system.

Why This Step Matters

This step matters because Spark must be running before I can load data, create Spark DataFrames, or build the recommender system. Verifying the Spark version also helps document the environment used for this project.

✓ Create the Movie Ratings Dataset in Spark

For this project, I am using the same synthetic movie ratings dataset from Project 4. Keeping the dataset consistent allows me to focus on the main purpose of this assignment, which is adapting the recommender system to Apache Spark and comparing it with the previous implementation.

The dataset includes users, movies, and ratings. Since Spark DataFrames work best in a long format, I will create the ratings data with one row for each user-movie-rating combination.

```
import pandas as pd
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Create users and movies
users = [f"User_{i}" for i in range(1, 51)]

movies = [
    "The Shawshank Redemption", "The Godfather", "The Dark Knight",
    "Pulp Fiction", "Forrest Gump", "The Matrix", "Inception",
    "The Lion King", "Titanic", "Jurassic Park", "Toy Story",
    "Finding Nemo", "Avengers: Endgame", "Goodfellas", "Gladiator",
    "The Social Network", "La La Land", "Get Out", "Interstellar", "Coco"
]

# Create synthetic ratings in long format
ratings_data = []

for user_id, user in enumerate(users, start=1):
    for movie_id, movie in enumerate(movies, start=1):
        rating = np.random.randint(1, 6)
        ratings_data.append((user_id, user, movie_id, movie, float(rating)))

# Convert to Pandas DataFrame first
ratings_pd = pd.DataFrame(
    ratings_data,
    columns=["user_id", "user", "movie_id", "movie", "rating"]
)

# Convert Pandas DataFrame to Spark DataFrame
ratings_spark = spark.createDataFrame(ratings_pd)

# Display the first few rows
ratings_spark.show(10, truncate=False)
```

	user_id	user	movie_id	movie	rating
	1	User_1	1	The Shawshank Redemption	4.0
	1	User_1	2	The Godfather	5.0
	1	User_1	3	The Dark Knight	3.0
	1	User_1	4	Pulp Fiction	5.0
	1	User_1	5	Forrest Gump	5.0
	1	User_1	6	The Matrix	2.0
	1	User_1	7	Inception	3.0

1	User_1	8	The Lion King	3.0
1	User_1	9	Titanic	3.0
1	User_1	10	Jurassic Park	5.0

only showing top 10 rows

Interpretation

The movie ratings dataset was successfully created and converted into a Spark DataFrame. Each row represents a single user-movie-rating combination, making the data suitable for Spark's distributed processing environment. Displaying the first ten rows confirms that the data was loaded correctly and is ready for the recommender system.

Why This Step Matters

Creating the dataset in Spark allows the recommender system to use Spark DataFrames instead of relying only on traditional Pandas DataFrames. This is important because Spark DataFrames are designed to support larger data processing workflows and connect this project to the distributed computing focus of Project 5.

Examine the Spark DataFrame

Before building the recommender system, I want to verify that the Spark DataFrame was created correctly. Reviewing the structure, number of records, and column information helps confirm that the dataset is ready for analysis and reduces the likelihood of errors later in the project.

```
# Display the schema
ratings_spark.printSchema()

# Count the total number of records
print("Total ratings:", ratings_spark.count())

# Display the first five rows
ratings_spark.show(5, truncate=False)
```

```
root
 |-- user_id: long (nullable = true)
 |-- user: string (nullable = true)
 |-- movie_id: long (nullable = true)
 |-- movie: string (nullable = true)
 |-- rating: double (nullable = true)
```

Total ratings: 1000

	user_id	user	movie_id	movie	rating
1	User_1	1	The Shawshank Redemption	4.0	
1	User_1	2	The Godfather	5.0	
1	User_1	3	The Dark Knight	3.0	
1	User_1	4	Pulp Fiction	5.0	
1	User_1	5	Forrest Gump	5.0	

only showing top 5 rows

Interpretation

The Spark DataFrame was created successfully with five columns representing the user, movie, and rating information. The dataset contains 1,000 ratings, which matches the expected 50 users rating 20 movies each. Reviewing the schema and sample records confirms that the data was loaded correctly and is ready for building the recommender system.

Why This Step Matters

Examining the Spark DataFrame helps verify that the data has been imported correctly before any analysis begins. Confirming the schema, record count, and sample data reduces the risk of carrying errors into later stages of the recommender system and provides confidence that the Spark environment is working as expected.

Prepare Data for Item-Based Collaborative Filtering

To build an item-based recommender system in Spark, the data must be structured so that similarity can be calculated between movies. This requires transforming the dataset into a user-item rating matrix format, where rows represent users, columns represent movies, and values represent ratings.

Although Spark works efficiently with long-format data, item-based similarity calculations require restructuring the data so that each movie can be compared against every other movie based on user ratings.

```
from pyspark.sql.functions import col

# Create user-item matrix using pivot
ratings_matrix = ratings_spark \
    .groupBy("user") \
    .pivot("movie") \
    .avg("rating")

ratings_matrix.show(5)
```

user	Avengers: Endgame	Coco	Finding Nemo	Forrest Gump	Get Out	Gladiator	Goodfellas	Inception	Interstellar	Jurassic Park
User_44	1.0	3.0	4.0	3.0	2.0	1.0	4.0	4.0		5.0
User_38	1.0	2.0	4.0	3.0	2.0	3.0	3.0	3.0		5.0
User_10	4.0	4.0	5.0	2.0	1.0	4.0	2.0	5.0		5.0
User_9	2.0	1.0	1.0	3.0	1.0	4.0	4.0	4.0		5.0
User_11	3.0	1.0	4.0	4.0	4.0	3.0	2.0	5.0		1.0

only showing top 5 rows

```
ratings_matrix = ratings_matrix.orderBy("user")
ratings_matrix.show(5)
```

user	Avengers: Endgame	Coco	Finding Nemo	Forrest Gump	Get Out	Gladiator	Goodfellas	Inception	Interstellar	Jurassic Park
User_1	5.0	4.0	3.0	5.0	5.0	4.0	2.0	3.0		1.0
User_10	4.0	4.0	5.0	2.0	1.0	4.0	2.0	5.0		5.0
User_11	3.0	1.0	4.0	4.0	4.0	3.0	2.0	5.0		1.0
User_12	4.0	5.0	1.0	1.0	4.0	1.0	4.0	2.0		5.0
User_13	4.0	2.0	4.0	3.0	5.0	1.0	3.0	4.0		2.0

only showing top 5 rows

Interpretation

The dataset was successfully converted into a user-item matrix format where each row represents a user and each column represents a movie. Missing ratings were filled with zero to allow similarity calculations between movies. The data is now structured in a way that enables comparison of movie rating patterns across users.

The data was also sorted by user to improve readability when inspecting the output, although this does not affect the recommender system logic.

Why This Step Matters

This step is necessary because item-based collaborative filtering requires comparing items (movies) based on user behavior. Reshaping the data into a matrix allows these comparisons to be computed efficiently.

In larger real-world systems, this transformation becomes more complex due to data size and sparsity. However, the same structure is commonly used in production recommender systems where distributed computing frameworks such as Spark become more useful.

✓ Compute Item-to-Item Similarity

In this step, I compute the similarity between movies based on user ratings. This is the core of the item-based collaborative filtering approach.

Instead of directly comparing users, I compare movies by treating each movie as a vector of user ratings. The similarity between these vectors helps identify which movies are most alike based on user behavior.

For this implementation, I use cosine similarity, which is commonly used in recommender systems because it measures the angle between two vectors rather than their magnitude.

```
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# Your user-movie matrix
user_movie_matrix = user_movie_matrix.fillna(0)

# Compute cosine similarity between movies (columns = movies)
movie_similarity = pd.DataFrame(
    cosine_similarity(user_movie_matrix.T),
    index=user_movie_matrix.columns,
    columns=user_movie_matrix.columns
)

movie_similarity.head()
```

movie	Avengers: Endgame	Coco	Finding Nemo	Forrest Gump	Get Out	Gladiator	Goodfellas	Inception	Interstellar	Jurassic Park	La La Land
Avengers: Endgame	1.000000	0.844359	0.844681	0.844449	0.840366	0.808999	0.807094	0.819701	0.812099	0.790626	0.8401
Coco	0.844359	1.000000	0.813482	0.823982	0.843029	0.793177	0.857655	0.889010	0.862520	0.847231	0.8025
Finding Nemo	0.844681	0.813482	1.000000	0.795616	0.753683	0.816113	0.782489	0.823961	0.763890	0.801039	0.8057
Forrest Gump	0.844449	0.823982	0.795616	1.000000	0.811577	0.844182	0.807089	0.845860	0.781561	0.825011	0.8604
Get Out	0.840366	0.843029	0.753683	0.811577	1.000000	0.788370	0.821676	0.850668	0.762252	0.793978	0.8091

Next steps: [Generate code with movie_similarity](#) [New interactive sheet](#)

Start coding or [generate](#) with AI.

Interpretation

The cosine similarity between movies was successfully computed by comparing each movie based on how users rated them. Each movie is treated as a set of user ratings, and the similarity score measures how closely two movies match based on those ratings. Movies with higher similarity values are more likely to be recommended together.

Although Spark is used for data processing, cosine similarity is computed using Pandas for simplicity due to the small dataset size.

Why This Step Matters

This step is the core of the item-based recommender system because it determines how movies are related to one another based on user behavior. Instead of relying on raw ratings, the system now understands relationships between items.

In real-world recommender systems, such as Netflix or Amazon, this type of similarity calculation allows the system to recommend items based on patterns in user behavior rather than movie characteristics. As the number of users and items grows, distributed computing frameworks such as Spark make it possible to perform these calculations efficiently across multiple computers.

Generate Movie Recommendations

Using the item-to-item similarity matrix, I can now generate movie recommendations for a specific user. The idea is to identify movies the user has already rated highly and then find other movies that are most similar to those items.

In this model, “similar movies” does not refer to genre, storyline, or actors. Instead, similarity is based on user behavior. Movies are considered similar if they tend to be rated in a similar way by the same users.

This approach demonstrates how collaborative filtering can be used to suggest new content based on past user preferences.

```

import numpy as np

# Choose a user (we'll start simple with User_1)
target_user = "User_1"

# Get movies rated by the user
user_ratings = ratings_pd[ratings_pd["user"] == target_user]

# Filter only movies they liked (rating >= 4)
liked_movies = user_ratings[user_ratings["rating"] >= 4]["movie"].tolist()

print("Movies liked by user:", liked_movies)

# Generate recommendations
recommendations = pd.Series(dtype=float)

for movie in liked_movies:
    if movie in movie_similarity.columns:
        recommendations = recommendations.add(movie_similarity[movie], fill_value=0)

# Remove movies already seen
recommendations = recommendations.drop(labels=liked_movies, errors="ignore")

# Sort recommendations
top_recommendations = recommendations.sort_values(ascending=False).head(10)

top_recommendations

```

Movies liked by user: ['The Shawshank Redemption', 'The Godfather', 'Pulp Fiction', 'Forrest Gump', 'Jurassic Park',
0

movie	
Inception	9.213667
The Social Network	9.032402
Goodfellas	9.011515
The Matrix	8.959878
The Lion King	8.909011
Finding Nemo	8.870217
Titanic	8.860804
The Dark Knight	8.836427
Interstellar	8.733814

dtype: float64

Higher scores indicate stronger similarity between the user's preference profile and each recommended movie.

Interpretation

The recommender system generates movie suggestions for a specific user by first identifying movies the user has rated highly. It then uses the item-to-item similarity matrix to find other movies that have similar rating patterns across users.

The final output shows movies ranked by their overall similarity score, with higher values indicating a stronger relationship to the user's preferences. These recommendations are based on collaborative filtering rather than movie attributes such as genre or storyline.

Why This Step Matters

This step demonstrates the core value of a recommender system: generating personalized recommendations based on user behavior patterns. In real-world applications, this approach allows systems to learn relationships between items without relying on explicit metadata.

This method becomes especially powerful in large-scale systems where user interaction data is continuously growing and must be processed efficiently.

Comparison: Spark vs Original Implementation (Project 4)

In this project, I adapted my item-based collaborative filtering recommender system from a traditional Python implementation (Project 4) to Apache Spark.

Both implementations produce similar recommendation logic, but they differ significantly in how the computation is handled.

The original implementation used Pandas and operated on a single machine, which is efficient for small datasets but limited in scalability. In contrast, Spark is designed to distribute computations across multiple machines, making it more suitable for large-scale datasets.

Interpretation

The second part of the comparison shows that both Spark and the original Python implementation produce the same type of recommendations, but they reach the result in different ways.

The original approach uses a single computer to perform all calculations, while Spark prepares the same work in a way that could be distributed across multiple computers if needed.

Why This Step Matters

This step highlights why different computing approaches are needed depending on the size of the data.

For small datasets (for example, a few thousand movie ratings, a class assignment dataset, or a small customer list), a single computer using Pandas is usually enough. It is simple, fast, and easy to work with.

However, in real-world systems like Netflix, Amazon, or Spotify, the data can grow into millions or even billions of user interactions. At that scale, a single computer may run into limitations such as:

- running out of memory when holding large datasets in memory
- slower processing when performing repeated calculations on large data
- difficulty scaling when more data is added over time

Spark addresses this by splitting the workload across multiple machines, allowing large datasets to be processed more efficiently.

The tradeoff is that Spark introduces additional setup and system complexity compared to Pandas, but this complexity becomes worthwhile as data size increases.

Conclusion

This project demonstrated how an item-based collaborative filtering recommender system can be adapted from a traditional Python implementation to Apache Spark.

The Spark implementation follows the same recommendation logic as the original version but introduces a distributed computing framework that is designed to handle much larger datasets by dividing work across multiple machines.

For the dataset used in this project, which contains a small number of users and movie ratings, Spark does not provide a significant performance advantage. A single-machine solution such as Pandas is sufficient and simpler to implement.

However, Spark becomes necessary in real-world environments where recommender systems process millions or even billions of user interactions. In those cases, a single machine would be limited by memory and processing power, while Spark allows the workload to be distributed and scaled efficiently.

Overall, this project highlights the tradeoff between simplicity and scalability. The original implementation is easier to understand and build, while Spark introduces additional complexity but enables large-scale data processing that is essential in modern recommendation systems.

